

ClearMed: Executive Summary

1. Overview

ClearMed is a clinical document simplification platform that turns complex, jargon-heavy medical documents (discharge summaries, physician notes, and reports) into plain-language explanations a patient can actually understand. The system detects medical terminology in an uploaded or doctor-authored document, attaches a short, source-verified explanation next to each term, and lets the doctor review and approve the result before it ever reaches the patient. It is designed to sit inside existing hospital workflows rather than replace them: it already ships as both a patient-facing app, with its own accounts and document library, and a working microfrontend widget inside OpenMRS, one of the world's most widely deployed open-source electronic medical record systems.

ClearMed is currently a working prototype with real clinical validation: three physicians across two Israeli hospitals (Sheba Medical Center and Rambam Health Care Campus) reviewed the system's output against original clinical documents and confirmed it preserved medical accuracy while being understandable to a layperson. The company is pre-revenue, with the underlying software, term-detection engine, and OpenMRS integration already functional today.

2. The Problem & Target Audience

The problem

Patients routinely leave medical encounters holding documents they cannot understand. Medical language is written for doctors, not for the people it describes, and that gap has two costs:

- **For patients:** not understanding a diagnosis, a discharge instruction, or a test result increases anxiety and undermines informed decision-making about one's own care.
- **For hospitals:** when patients do not understand their instructions, they call back with questions, miss follow-up steps, or return to the hospital when a problem could have been avoided. Every one of those outcomes consumes clinical staff time that is already scarce.

Validation of the need

ClearMed's own user research quantifies this gap rather than assuming it:

- In a patient survey, **more than 61% of respondents rated medical terminology a 4 or 5 out of 5 for difficulty to understand**, and the negative effects reported included reduced ability to make informed decisions and increased anxiety.
- **70% of respondents said it was important to them to understand their medical discharge papers.**

Beyond the survey, ClearMed ran a direct clinical validation experiment: a physician-written medical document was translated into a simplified, patient-friendly version, and both versions were sent side by side to practicing physicians for review. Reviewers included a doctor at Sheba Medical Center, the Head of Internal Medicine at Rambam Health Care Campus, and a specialist physician. Across all three:

- **Clinical accuracy** was confirmed: the simplified version was judged fully or highly accurate, with no critical medical information lost (one reviewer noted minor generalization of specific details, none flagged missing risk information).
- **Willingness to sign** was strong: all three said they would be willing to actively review and approve the simplified document alongside the official one before it reaches a patient, one described this willingness as unconditional.
- **Time savings** were acknowledged by all three, particularly for reducing explanation time during discharge and follow-up calls, though one reviewer noted some patients will still prefer a face-to-face conversation in addition.

Target audience

ClearMed has two audiences that sit on either side of the same transaction:

- **The buyer:** hospitals and clinics, specifically the departments and IT/EMR teams responsible for discharge processes and patient communication, and by extension EMR platforms themselves (ClearMed's OpenMRS widget makes the EMR vendor and its hospital network a natural second buyer path).
- **The beneficiary:** patients receiving care, who are the ones who actually read and rely on the simplified document.

3. The Product & User Value

ClearMed has two real entry points into the same underlying document-simplification engine: a self-service patient app, and a clinician-reviewed flow embedded directly in the hospital's EMR. They are genuinely different workflows, not the same flow with two front doors, so they are described separately below.

Patient app (self-service) 1. The patient registers or logs in (a JWT session in an HttpOnly cookie; the password itself is never stored, only an argon2 hash). Every new account starts with five default folders already in place ("Lab Results," "Imaging," "Prescriptions," "Surgery," "General"), so a document has somewhere to live from the very first upload. 2. The patient uploads a document (PDF, JPG, or PNG) into a folder. Scanned or photographed documents go through an OCR step automatically. 3. The system scans the extracted text and detects medical terms and phrases it recognizes, using the same verified medical-term dictionary described below. English content is built from MedlinePlus, the U.S. National Library of Medicine's consumer health information service (NIH); Hebrew content comes from Infomed (infomed.co.il), an Israeli medical information site, matched concept by concept to its English counterpart rather than machine-translated from it (see Section 9.3 for exactly how). 4. The patient themselves reviews the detected terms and chooses which ones to see explained, then triggers simplification. No clinician reviews or approves this particular flow, it is entirely self-service. 5. The simplified result is saved back onto that same document in the patient's own library, so it can be reopened, re-simplified with a different term selection, or annotated with a personal note at any time, not just viewed once.

OpenMRS widget (clinician-reviewed) 1. A document is authored directly inside a patient's chart through the OpenMRS widget, and scanned against the same verified dictionary. 2. Each detected term is matched to a short, plain-language explanation drawn from that dictionary, never invented on the spot. 3. The doctor reviews which terms should be explained and can approve, adjust, or re-run the simplification before anything is shown to the patient. This keeps a human in the loop for every document that reaches a patient through this channel. 4. The patient receives the simplified document, and can also receive it as a scannable QR code that opens a mobile-friendly view of just their explanation, without requiring them to create an account or carry a printed page.

Value to hospitals

Faster discharge conversations, fewer repetitive follow-up calls answering the same terminology questions, and a reviewable, sign-off-ready artifact that fits into an existing clinical documentation habit rather than replacing it.

Value to patients

A document written in language they can actually use to understand their own condition and instructions, without losing the clinical substance of what their doctor said, and delivered in a form (a phone-scannable link) that travels with them after they leave the hospital. For patients using the app directly, that record also persists in their own account across visits, not just for the length of one session.

4. Innovation & Key Advantages

Verified-source term explanation. ClearMed's explanations are not written on demand by a language model at the moment a patient opens their document. Every explanation stored in the system's dictionary is drawn, sentence for sentence, from MedlinePlus source text. This removes the single biggest risk in AI-assisted patient communication: a model inventing or subtly distorting medical content.

A paragraph-shortening algorithm built to prevent hallucination structurally, not just prompt it away. Long MedlinePlus explanation paragraphs are shortened into single-sentence explanations by a process that treats an AI model strictly as a *selector*, never as a *writer*. The long paragraph is split into a numbered list of candidate sentences. A model (GPT-4o-mini) is given those candidates alongside a detailed decision hierarchy (does the sentence stand alone without prior context, does it explain the term itself rather than a related fact, does it convey real understanding rather than a shallow dictionary definition, and so on) and is constrained, via a strict JSON-schema response format, to return only the index number of one candidate sentence. The system then pulls that exact sentence back out of its own original list. The model is structurally incapable of introducing new text into the final output, because its only valid output is an integer. If the call fails or returns an invalid index, a deterministic keyword-based fallback (looking for phrases like "is a," "means," or "refers to") selects a sentence instead, with no AI call at all. This selection step also runs once, offline, when the dictionary is built, not per patient request, so it adds no latency or failure risk to the live document pipeline. Non-English versions of the same short explanation are produced by deterministic machine translation (DeepL), not a second AI generation step.

Doctor-in-the-loop, where a clinician is in the workflow at all. In the OpenMRS hospital channel specifically, the product is built around a review-and-approve step, not a fully automated pipeline, which is exactly what earned the "willing to sign" response from the validating physicians.

A persistent, self-service record, not a one-off session. The patient app gives a patient their own account and document library, organized into folders, five of them seeded automatically on signup, so a document a patient uploads and simplifies stays there to reopen, re-simplify with a different term selection, or annotate with a personal note, rather than existing only for the length of one browser session or one QR scan.

Distribution through the EMR itself. Most patient-communication tools compete for a hospital's attention as a standalone app. ClearMed already ships a working microfrontend widget for OpenMRS that appears as a tab directly inside a doctor's existing patient chart, calling the same backend used by the standalone tool. This is a meaningfully different, and harder to replicate quickly, distribution path than a consumer-facing app competing for a download.

Multilingual by construction. English and Hebrew are supported today, with additional languages (Arabic, Russian) on the near-term roadmap, which matters directly for hospitals serving linguistically diverse patient populations.

Deterministic translation reaches far beyond the languages the dictionary was built for. ClearMed's core medical dictionary is curated and verified in a small, fixed set of languages, English and Hebrew. But the final explanation document a patient receives is not limited to those languages: it can be translated on demand into any of the 100-plus languages supported by the Google Cloud Translation API, through a separate, deterministic translation step that only ever operates on already-verified, already-approved text, never on the raw medical dictionary itself. Because this is deterministic machine translation rather than generative rewriting, it carries the same **no-hallucination** guarantee as the rest of the pipeline. In practice, this means a patient who speaks a language the dictionary was never built for, Amharic, for example, spoken by Israel's Ethiopian-Israeli community, can still receive their explanation in their own language today, without waiting for that language to be added to the curated dictionary. This gives ClearMed a direct answer for **linguistic minorities** inside Israel's own patient population who are otherwise underserved by hospital communication.

5. Competitors & Alternatives

The closest points of comparison fall into two groups: patient-facing translation tools, and doctor-facing documentation tools that solve an adjacent but different problem.

- **Vital** (a health-tech company founded by the creator of Mint.com, having raised roughly \$25 million in a Series B round) launched an AI "Doctor-to-Patient Translator" that turns discharge summaries and clinical notes into roughly 5th-grade reading level text, but generates an entire new text via generative AI rather than selecting from verified source sentences.

- **Nabla**, which raised a \$24 million Series B, addresses a related but distinct problem: it listens to a clinical consultation and generates the doctor's own note, prescriptions, and follow-up letters. It is a doctor-productivity tool for writing documentation, not a patient-facing simplification tool, and is included here because it is frequently mentioned in the same "AI in clinical documentation" conversation.
- A separate category of **generic medical translation and localization vendors** (language-translation platforms aimed at multilingual clinical content, not literacy simplification) solve a different problem entirely: translating language, not translating complexity, and are not direct competitors.

Where ClearMed differs

ClearMed's differentiation is structural, not just a matter of UI polish: it is embedded directly in the EMR a doctor already uses (via the OpenMRS widget) rather than asking doctors or patients to adopt a separate app, its explanations are anchored to a curated, verified medical dictionary rather than generated freely at request time, and its accuracy safeguard is architectural (constrained index-selection, never open-ended generation) rather than a matter of prompt engineering alone.

6. Target Market & Market Potential

A specific, directly addressable value pool is **preventable hospital readmissions**. In the United States alone, hospital readmissions cost an estimated \$17 billion to \$41 billion annually depending on the scope measured, with roughly one in five Medicare fee-for-service patients rehospitalized within 30 days of discharge, and research estimating that as many as 75% of readmissions are potentially preventable, frequently tied to inadequate communication of discharge instructions. ClearMed's core function sits directly inside that preventable slice.

OpenMRS gives ClearMed a concrete, already-built distribution beachhead. OpenMRS is deployed in more than 80 countries across more than 8,100 healthcare facilities, serving an estimated 16 to 22 million patients worldwide, heavily concentrated in low- and middle-income countries where OpenMRS is the dominant open-source EMR. Because ClearMed's OpenMRS widget already works today, expansion into that installed base is a matter of adoption and integration effort within an existing ecosystem, not a from-scratch market-creation exercise.

7. Go-to-Market Strategy

ClearMed's go-to-market is pilot-led and relationship-driven, building outward from validation the company has already earned rather than starting cold:

1. **Convert existing clinical relationships into paid pilots.** The physicians who validated the product at Sheba Medical Center and Rambam Health Care Campus are a warm starting point for formal hospital pilots in Israel, ClearMed's home market.
2. **Land within a department, then expand.** A single hospital department (a natural starting point is a high-discharge-volume department such as internal medicine) is easier to sign and prove value in than a hospital-wide rollout, and success in one department is the natural case study for expanding to others in the same institution.
3. **Use the OpenMRS widget as a channel, not just a feature.** Because the integration already exists, ClearMed can pursue distribution through the OpenMRS implementer and partner ecosystem itself, reaching hospitals that have already made the EMR decision and are looking for add-on capability, rather than competing for a hospital's attention as an entirely new vendor relationship.

8. Team & Responsibilities

Tal Gershanov, Co-Founder and Backend & Widget Development. Yuval Bashan, Co-Founder and Backend & Frontend app Development.

9. Technical Architecture & Deep Dive

This section documents ClearMed's core, production document-processing system: its FastAPI backend, term-detection and translation logic, the patient app and its own persistence layer, the OpenMRS integration, and the AWS infrastructure all of it runs on today.

9.1 System Architecture

ClearMed's backend is a single FastAPI application. The original legacy and OpenMRS-facing code is organized into three internal layers, an API layer, a logic layer, and a data access layer, so that each layer only depends on the abstraction below it, not on a specific implementation. The patient app adds a second, parallel vertical alongside this, its own routers, its own orchestration service, and its own persistence, described in the note after the diagram below.

The system's AWS deployment topology, including exactly where each of these stores physically lives, is described in Section 9.2.

To be precise about where each layer's responsibility actually starts and stops on the legacy/OpenMRS path, since it is easy to assume the API layer touches the database: the FastAPI layer (`server/api.py`) is purely a request handler, it parses the incoming request, calls exactly one Logic-layer function, and serializes that function's return value back to the client. It never queries SQLite itself (its only interaction with `DAL/` anywhere in that part of the codebase is a one-time `shares_db.init_schema()` call at process startup, not a per-request data operation). Every actual database read and write, looking up a term's explanation, building the trie from the alias table, creating or reading a QR share, is issued from inside the Logic layer (`medical_term_detector.py`, `medical_term_trie.py`, `document_shares.py`), always through the DAL's abstract interface rather than by importing SQLite directly. The Logic layer is the sole owner of when and how persistence happens on this path; the API layer only decides which Logic function to call.

The patient app is not a fourth example of that same layering, and it is worth saying so plainly rather than forcing it into the same shape. `webapp/documents/service.py` calls the exact same shared Logic-layer functions the legacy `/analyse` and `/translate` routes use (`detect_terms_with_explanations`, `apply_translations`), reused, never duplicated, but it manages its own persistence directly via SQLAlchemy (`webapp/core/database.py`), a connection explicitly commented in the code as one that "must never be shared with `DAL/db.py`." The patient app never touches the `DAL/` package at all; `DAL/` remains scoped entirely to `clearmed.db` and `shares.db`. So there are, accurately, two different data-access patterns living side by side in one FastAPI process: the original strict API-to-Logic-to-DAL layering for the term dictionary, and a separate ORM-based vertical for the patient app's own users, folders, and documents, which happens to reuse the first system's term-detection logic without adopting its persistence abstraction.

Maintainability through layering and polymorphism. The 3-layer structure is not just an organizational convention, it is enforced through interfaces that let concrete implementations vary without touching the layers above them. Two examples:

- The logic layer's term detectors share a common interface (`logic/term_detectors/base.py`), with a separate concrete implementation for English (`english.py`) and Hebrew (`hebrew.py`, which additionally strips Hebrew grammatical prefixes before matching). A factory (`factory.py`) selects the correct detector at runtime. Adding a third language means adding one new class that satisfies the existing interface, not modifying the API or DAL layers at all.
- The data access layer is defined as an abstract interface (`DAL/interface.py`) that the logic layer depends on, with a concrete SQLite implementation (`DAL/db.py`) satisfying that interface. The logic layer never imports SQLite directly, so the storage engine could be swapped without changing any detection or translation code. The patient app's own storage layer follows the identical pattern one level up: `webapp/storage/base.py` defines an abstract `StorageBackend`, with a `LocalStorageBackend` for development and an `S3StorageBackend` for production (Section 9.2), selected at runtime by a single env var, with no other code aware of which one is active.

Two distinct translation mechanisms exist in this system, and it is worth being precise about which one runs when. DeepL (`server_init/translation_service.py`, the dotted DeepL edge in the diagram above) translates the dictionary's own short explanation into the small, fixed set of languages the term dictionary is built for, English and Hebrew today, offline, once, at database-build time. Google Cloud Translation API (`logic/document_translation.py`, not shown above since it sits outside the dictionary-build path) is the separate, runtime mechanism behind the any-language document translation described in Section 4: it translates the final, already-assembled explanation document into any of the 100-plus languages Google Translate supports, on demand, per request, regardless of which languages the underlying dictionary was built for. Both are deterministic machine translation, never a generative rewrite, so neither introduces the hallucination risk a second LLM pass would.

9.2 AWS Deployment Architecture

ClearMed runs today on a single AWS EC2 instance in `eu-north-1`.

The IAM instance role is deliberately the only path from this instance to any AWS-managed service: it grants exactly `s3:GetObject/PutObject/DeleteObject/ListBucket` on the `clearmed-uploads-yuval2026` bucket, and no AWS access keys exist anywhere in the deployed `.env`, matching the storage backend's own design (`webapp/storage/s3.py`, an `S3StorageBackend` that reads credentials from nowhere in config and relies entirely on that role, per Section 9.1's polymorphism note). The RDS Postgres instance (`clearmed-db`, `db.t4g.micro`) is not publicly accessible; it is reachable only via a security-group-to-security-group rule scoped to this EC2 instance, not a public IP allowlist. Both `clearmed.db` and `shares.db` remain local SQLite files on the instance's own disk rather than managed AWS services, which is exactly why they carry the single-instance caveats noted in Section 9.3. This is a single point of failure by construction, one instance, one region, no load balancer, a known near-term infrastructure risk rather than an oversight.

9.3 Database Schema

ClearMed uses three separate, physically distinct data stores, one Postgres database and two SQLite files, and none of them is ever merged or queried across another.

This is the patient app's own database: Amazon RDS PostgreSQL in production (`clearmed_app`), a local Docker Postgres container in development (`docker-compose.yml`), read and written exclusively via SQLAlchemy (`webapp/*/models.py`), migrated with Alembic. `USERS` holds one row per registered patient; the password itself is never stored, only an argon2 hash. `FOLDERS` is a self-referencing tree (`parent_folder_id`); every new user gets five default folders automatically ("Lab Results," "Imaging," "Prescriptions," "Surgery," "General"), and deleting a folder that still has sub-folders is blocked at the database level (`ON DELETE RESTRICT`) rather than silently orphaning them. `DOCUMENTS` carries two distinct groups of columns: identity and storage (where the file physically lives, `storage_key` is the S3 object key when `STORAGE_BACKEND=s3`, or a local filename otherwise) and the ClearMed pipeline itself (extraction, analysis, and simplification status, each independently trackable, plus a freeform `notes` field that is deliberately independent of that pipeline). Deleting a user cascades to their folders and documents; deleting a non-empty folder does not.

Two caveats are worth stating plainly rather than glossing over. The in-memory trie described in Section 9.4 is built once per process at startup and never scales beyond that one instance's memory today; a second app instance behind a load balancer would rebuild its own independent copy rather than share one. `shares.db` is similarly single-instance-safe only: if a second app instance is ever introduced, that store would need to move into the shared Postgres database instead of staying as a local SQLite file.

The verified medical dictionary and the ephemeral share store, ClearMed's other two data stores, are both SQLite databases, each with a small, purpose-built schema.

`CONCEPTS` holds one row per medical concept, currently roughly 1,017 concepts, all anchored to an English MedlinePlus source. `EXPLANATIONS` holds one row per concept per supported language, carrying both the full explanation and the shortened, selected explanation described in Section 4; 413 of the 1,017 concepts also have a Hebrew row today (see the sourcing note below for where that Hebrew text actually comes from). `TERM_ALIASES` holds every synonym and alternate phrasing

for a concept (roughly 900 additional English aliases on top of the primary terms) and is the exact table used to build the search trie described below. `SHARED_DOCUMENTS` is intentionally disconnected from the other tables: it lives in its own SQLite file, stores a denormalized snapshot of an already-approved explanation for the QR-code handoff flow, and rows are physically deleted after a 5-minute expiry window, since it is designed to hold sensitive content only as briefly as necessary.

Where the English and Hebrew explanation text actually comes from. These two languages are not produced the same way, and the difference matters for accuracy. English `simple_explanation` text is parsed directly from MedlinePlus Health Topic XML (`data_preparation/convert_medline_xml_to_json.py`), and it is this English concept set that defines the canonical `concept_id` space; every other language attaches to an existing English concept, none is created independently. Hebrew `simple_explanation` text is not a machine translation of that English text: it is scraped from Infomed (infomed.co.il), a Hebrew-language Israeli medical information site, by a separate offline script (`data_preparation/scrape_infomed_to_json.py`) that matches each Infomed page to its corresponding English concept, either by an exact match against one of that concept's existing English names or synonyms, or, for disease pages that carry no English name at all, by matching the page's own URL slug, and only then stores Infomed's natively-written Hebrew paragraph as that concept's Hebrew explanation. A page with no matching English concept is skipped entirely rather than guessed at. The one-sentence `short_explanation`, by contrast, is deliberately handled differently for Hebrew: rather than running a second, independent AI selection pass over the Hebrew paragraph, the system takes the already-selected English short explanation (the output of the GPT-4o-mini selection algorithm in Section 4) and machine-translates it into Hebrew via DeepL - a *deterministic* translation model. This keeps the short explanation's selection logic and quality bar identical across languages, while the longer explanation each language shows a patient is still written natively for that language rather than reading like translated English.

Why `EXPLANATIONS` and `TERM_ALIASES` each carry both a primary key and a foreign key. These are two different keys doing two different jobs, and they are not in tension: the primary key (`explanation_id`, an auto-incrementing integer; `alias_text`, the alias string itself) uniquely identifies that row within its own table, while the foreign key (`concept_id` in both tables) says which `CONCEPTS` row that row belongs to. A table is free to have its own identity and still be a dependent of another table at the same time. And to directly answer the dependency question: yes, these are real, enforced foreign keys, not just a documentation convention. Both columns are declared with an explicit `REFERENCES concepts (concept_id)` clause in the schema (`server_init/db_operations.py`), and the database connection explicitly turns on `PRAGMA foreign_keys = ON` (`DAL/db.py`) so SQLite actually enforces it. In practice this means a row can never be inserted into `EXPLANATIONS` or `TERM_ALIASES` for a `concept_id` that does not already exist in `CONCEPTS`, and a `CONCEPTS` row cannot be deleted while dependent `EXPLANATIONS` or `TERM_ALIASES` rows still reference it. `TERM_ALIASES` additionally uses `alias_text` itself, not an auto-incrementing id, as its primary key, since every alias string across the entire dictionary must be unique (it is the exact key the trie is built from), and enforcing that uniqueness at the database level is more direct than reconstructing it in application code.

9.4 Prefix Trees (Trie) Structure

ClearMed uses an in-memory trie to find medical terms and phrases inside a block of uploaded or authored text. The full tree is built from roughly 1,400 to 2,000 phrases (all English and Hebrew concept names plus their aliases), so it is not practical to show in full. The diagram below shows the root, a few of its first-level branches, and representative leaf nodes several levels down; the "..." nodes stand in for the hundreds of other branches the real tree contains.

9.5 Tree Implementation Details

Two clarifications are worth stating precisely, since both correct assumptions that are easy to make about a system described as using "prefix trees":

Every node is a hash map, which is what makes each step of a search $O(1)$. Each `TrieNode` is a plain object holding exactly three fields: a `children` dictionary keyed by the next word, a boolean `is_end` flag marking whether that path completes a real term, and, if `is_end` is true, the concept ID that term maps to. The root of the tree is itself a `TrieNode`, so it starts with a

`children` dictionary too, there is no special-cased root structure. Descending one level, from the root into its children, or from any node into its children, is therefore a single Python dictionary lookup (`children[word]`), which is $O(1)$ on average regardless of how many other branches exist at that level or how large the overall dictionary is. Inserting a phrase during the offline database-build step works the same way in reverse: walk word by word, creating a new `TrieNode` (and therefore a new empty `children` dict) wherever a needed branch does not exist yet, then set `is_end = true` and attach the concept ID on the final node.

This is a word-level trie, not a character-level trie. Each node's children are keyed by whole normalized tokens (words like "type," "2," or "diabetes"), not by individual letters. This means the tree is really an index over multi-word medical phrases, and its branching factor at any node is the number of distinct next-words seen in the source data at that point, not a fixed alphabet size like 26.

The search algorithm itself is a **longest-match, non-overlapping scan**: starting at each word position in the document, it walks as far down the tree as the document's words continue to match existing children, remembers the longest match found (marked `is_end = true`), and on a match jumps past the entire matched phrase before continuing, so overlapping partial matches are never double-counted. This is conceptually close to the Aho-Corasick string-matching algorithm, with one deliberate simplification: Aho-Corasick precomputes "failure links" so that after a partial mismatch it can jump directly to the next-best position in the tree without restarting; ClearMed's implementation instead restarts the walk from the root at the next word position after a miss. Given the tree's modest size (a few thousand phrases, each only a few words long), this simplicity costs negligible practical performance while keeping the implementation easy to reason about and maintain, consistent with the maintainability priority described in Section 9.1.